

FastAPI Philosophy

FastAPI Philosophy, Setup, and First Demo

1. Introduction

This section sets the goal:

You will learn:

- **Why FastAPI exists (its philosophy)**
- **Why it's different from other frameworks**
- **How to install it**
- **How to build your first API**

 Think of this as moving from:

Understanding → Setup → Practical use

2. Philosophy of FastAPI

Why was FastAPI created?

Before FastAPI, developers used frameworks like:

- Flask
- Django

These are powerful, but:

- Require more setup
- Need extra libraries for validation
- Documentation is not automatic

 FastAPI was created to fix these problems.

Core Philosophy

1. Speed (Performance)

FastAPI is **extremely fast**, almost like Node.js.

 Why?

- Built on modern technologies:
 - **ASGI (Asynchronous Server Gateway Interface)**
 - Async programming support
-

Simple idea:

It can handle **many users at the same time efficiently**

2. 🛠️ Clean & Simple Code

FastAPI encourages:

- Writing less code
 - Writing clean code
 - Avoiding unnecessary complexity
-

📄 Example Idea:

Instead of writing:

- Separate validation logic
- Separate documentation
- Extra configuration

👉 FastAPI does it automatically

3. 🛡️ Built-in Validation

FastAPI uses **type hints** (Python feature):

```
def add(a: int, b: int):
```

👉 This tells FastAPI:

- a must be integer
- b must be integer

If wrong input comes → FastAPI rejects it automatically

4. 📄 Automatic Documentation

FastAPI automatically generates:

- Swagger UI (interactive API docs)
- ReDoc

👉 You can test your API directly in browser

5. 🏠 Perfect for Modern Apps

FastAPI is ideal for:

- APIs
 - Machine Learning deployment
 - Microservices
-

3. Second Philosophy (Why FastAPI is Fast to Code)

This is not about speed of execution

👉 It's about **developer productivity**

Why FastAPI is fast to write?

1. Smart Defaults

FastAPI:

- Assumes good defaults
- Reduces configuration

👉 You write less setup code

2. Less Repetition

In older frameworks:

- You repeat code for validation, parsing, etc.

FastAPI:

- Does it automatically
-

3. Type Hints = Multiple Benefits

With one line:

name: str

You get:

- Input validation
- Auto documentation
- Error handling

👉 One feature → multiple benefits

4. Based on Powerful Tools

FastAPI internally uses:

- **Pydantic** → for validation
- **Starlette** → for performance

👉 You get power without complexity

Result:

- Faster development
 - Fewer bugs
 - Cleaner structure
-

4. Installation and Code Demo

Now we move to practical part.

Step 1: Install FastAPI

Open terminal and run:
pip install fastapi uvicorn

What are these?

- **fastapi** → framework
 - **uvicorn** → server to run your API
-

Step 2: Create First File

Create a file:
main.py

Step 3: Write Code

from fastapi import FastAPI

app = FastAPI()

```
@app.get("/")
def home():
    return {"message": "Hello FastAPI"}
```

Understanding the Code

1. Create app

```
app = FastAPI()
```

 This initializes your API

2. Create endpoint

```
@app.get("/")
```

 This means:

- When someone visits /
 - Run the function below
-

3. Function response

```
return {"message": "Hello FastAPI"}
```

 API sends JSON response

Step 4: Run Server

In terminal:

uvicorn main:app --reload

Explanation:

- main → file name
 - app → FastAPI object
 - --reload → auto restart on changes
-

Step 5: Open in Browser

Go to:

<http://127.0.0.1:8000/>

 Output:

```
{"message": "Hello FastAPI"}
```

Step 6: API Documentation (Very Important)

Open:

<http://127.0.0.1:8000/docs>

 You'll see:

- Interactive UI
 - Test endpoints
 - Input/output structure
-

Why this is powerful:

Without extra effort, you get:

- Working API
 - Documentation
 - Validation
-

Real-World Example

Imagine:

You are building a **student result system**

Instead of:

- Writing backend from scratch

You can:

- Create FastAPI endpoints:
 - /students
 - /results
 - /login
-

👉 Apps can connect to these APIs easily

🧠 Final Summary

🔑 FastAPI Philosophy:

- Fast to run ⚡
 - Fast to write 🍷
 - Clean and simple 🧼
-

🔑 Key Features:

- Automatic validation
 - Auto documentation
 - High performance
 - Beginner-friendly
-

🔑 Workflow:

1. Install FastAPI
 2. Create app
 3. Define routes
 4. Run server
 5. Test in browser
-

🎯 Memory Trick

👉 **FastAPI = Less Code + More Power**
